

**Федеральное государственное автономное образовательное учреждение
высшего образования**

"Национальный исследовательский университет

"Высшая школа экономики"

Московский институт электроники и математики им. А. Н. Тихонова

Департамент компьютерной инженерии

Отчёт по дисциплине

«Распределённые базы данных и сетевые вычисления»

Потоки выполнения. Синхронизация

Студент:

Абрамов Илья Александрович

Группа: МКС254

Преподаватель: Байбикова Т.Н.

1 Задание

Изучить теоретический материал для практики и разобрать прилагаемые примеры. Создать свои примеры, в которых содержатся: класс Thread, интерфейс Runnable. Поработать с приоритетами и создать свои примеры, в которых потокам назначаются приоритеты. Создайте примеры, в которых потоки синхронизируются.

Подготовить отчет, включить в него свои примеры (листинги) с кратким описанием.

2 Разработанная программа

Всего было создано 16 файлов и 5 перенесено:

- src/common/ - классы самолетов из прошлой работы
 - Aeroplane.java
 - Engine.java
 - Flyable.java
 - Maintainable.java
 - Vehicle.java
- App.java - запускает все демонстрации для заданий
- Задание 1 (Класс Thread)
 - PlaneMaintenanceThread.java - поток обслуживания
 - FlightSimulatorThread.java - поток симуляции полета
 - ThreadDemo.java – демонстрация
- Задание 2 (Интерфейс Runnable)
 - RefuelingTask.java - задача заправки
 - BoardingTask.java - задача посадки пассажиров
 - PreFlightCheckTask.java - задача предполетной проверки
 - RunnableDemo.java – демонстрация
- Задание 3 (Приоритеты потоков)
 - EmergencyLandingTask.java - критическая задача
 - RoutineMaintenanceTask.java - обычная задача
 - DataLoggingTask.java - фоновая задача
 - PriorityDemo.java – демонстрация
- Задание 4 (Синхронизация)
 - RunwayController.java - контроллер взлетной полосы
 - FuelTank.java - топливный резервуар
 - UnsafeCounter.java - пример race condition
 - SynchronizationDemo.java - демонстрация

3 Листинг

3.1 Файл PlaneMaintenanceThread.java

3.2 Файл FlightSimulatorThread.java

3.3 Файл ThreadDemo.java

3.4 Файл RefuelingTask.java

3.5 Файл BoardingTask.java

3.6 Файл PreFlightCheckTask.java

3.7 Файл RunnableDemo.java

```
import common.Aeroplane;
import common.Engine;

/**
 * Демонстрация работы с потоками через интерфейс Runnable
 *
 * Задание 2: Создание потоков с использованием интерфейса Runnable
 *
 * Ключевые концепции:
 * 1. Реализация интерфейса Runnable
 * 2. Создание потоков через new Thread(Runnable)
 * 3. Повторное использование задач в разных потоках
 * 4. Использование лямбда-выражений
 * 5. Сравнение с подходом через наследование Thread
 */
public class RunnableDemo {

    public static void main(String[] args) {
        System.out.println("=".repeat(80));
        System.out.println("ЗАДАНИЕ 2: ДЕМОНСТРАЦИЯ РАБОТЫ С ИНТЕРФЕЙСОМ
RUNNABLE");
        System.out.println("=".repeat(80));
        System.out.println();

        // Создаем самолеты для демонстрации
        Aeroplane boeing737 = new Aeroplane(
            "Boeing", "737-800", 2015, 850.0,
            189, 35.8, new Engine("CFM56-7B", 27300)
        );

        Aeroplane airbusA320 = new Aeroplane(
            "Airbus", "A320", 2018, 840.0,
            180, 35.8, new Engine("CFM56-5B", 27000)
        );
    }
}
```

```

Aeroplane boeing777 = new Aeroplane(
    "Boeing", "777-300ER", 2020, 905.0,
    396, 64.8, new Engine("GE90-115B", 52000)
);

System.out.println("Создано 3 самолета для демонстрации работы с
Runnable\n");

//
=====
// ПРИМЕР 1: Базовое использование Runnable
//
=====
System.out.println("\n" + "=".repeat(80));
System.out.println("ПРИМЕР 1: Базовое использование Runnable");
System.out.println("=".repeat(80));
System.out.println("Демонстрация: создание задач через реализацию
Runnable");
System.out.println("Запускаем задачи заправки и посадки параллельно\n");

// Создаем задачи
RefuelingTask refueling1 = new RefuelingTask(boeing737, 5000);
BoardingTask boarding1 = new BoardingTask(airbusA320, 180);

// Создаем потоки, передавая задачи
Thread refuelThread1 = new Thread(refueling1, "Refuel-Thread-1");
Thread boardThread1 = new Thread(boarding1, "Board-Thread-1");

// Запускаем потоки
refuelThread1.start();
boardThread1.start();

// Ждем завершения
try {
    refuelThread1.join();
    boardThread1.join();
    System.out.println("\n✓ Обе задачи завершены");
} catch (InterruptedException e) {
    System.err.println("Main thread interrupted");
    Thread.currentThread().interrupt();
}

//
=====
// ПРИМЕР 2: Повторное использование одной задачи в разных потоках
//
=====
System.out.println("\n" + "=".repeat(80));
System.out.println("ПРИМЕР 2: Повторное использование задачи");
System.out.println("=".repeat(80));

```

```

        System.out.println("Демонстрация: одна задача может быть запущена в
нескольких потоках");
        System.out.println("Запускаем одну задачу предполетной проверки в 3
потоках\n");

        // Создаем одну задачу
        PreFlightCheckTask checkTask = new PreFlightCheckTask(boeing777, "Chief-
Inspector");

        // Создаем несколько потоков с одной задачей
        Thread checkThread1 = new Thread(checkTask, "Check-Thread-1");
        Thread checkThread2 = new Thread(checkTask, "Check-Thread-2");
        Thread checkThread3 = new Thread(checkTask, "Check-Thread-3");

        // Запускаем все потоки
        checkThread1.start();
        checkThread2.start();
        checkThread3.start();

        try {
            checkThread1.join();
            checkThread2.join();
            checkThread3.join();
            System.out.println("\n✓ Все проверки завершены");
        } catch (InterruptedException e) {
            System.err.println("Main thread interrupted");
            Thread.currentThread().interrupt();
        }

        //
=====
        // ПРИМЕР 3: Использование анонимного класса
        //
=====

        System.out.println("\n" + "=".repeat(80));
        System.out.println("ПРИМЕР 3: Анонимный класс Runnable");
        System.out.println("=".repeat(80));
        System.out.println("Демонстрация: создание Runnable через анонимный
класс\n");

        Thread anonymousThread = new Thread(new Runnable() {
            @Override
            public void run() {
                String threadName = Thread.currentThread().getName();
                System.out.println "[" + threadName + "] Starting anonymous task
for " +
                                boeing737.getModel());
                try {
                    for (int i = 1; i <= 3; i++) {
                        System.out.println "[" + threadName + "] Step " + i + "
of 3");

```

```

        Thread.sleep(500);
    }
    System.out.println "[" + threadName + "] ✓ Anonymous task
completed");
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}
}, "Anonymous-Thread");

anonymousThread.start();

try {
    anonymousThread.join();
} catch (InterruptedException e) {
    System.err.println("Main thread interrupted");
    Thread.currentThread().interrupt();
}

//
=====
// ПРИМЕР 4: Использование лямбда-выражений (Java 8+)
//
=====

System.out.println("\n" + "=".repeat(80));
System.out.println("ПРИМЕР 4: Лямбда-выражения для Runnable");
System.out.println("=".repeat(80));
System.out.println("Демонстрация: современный способ создания Runnable
через лямбда\n");

// Простая лямбда
Thread lambdaThread1 = new Thread(() -> {
    String threadName = Thread.currentThread().getName();
    System.out.println "[" + threadName + "] Lambda task started");
    try {
        Thread.sleep(500);
        System.out.println "[" + threadName + "] Processing " +
airbusA320.getModel());
        Thread.sleep(500);
        System.out.println "[" + threadName + "] ✓ Lambda task
completed");
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}, "Lambda-Thread-1");

// Лямбда с захватом переменных
String destination = "New York";
Thread lambdaThread2 = new Thread(() -> {
    String threadName = Thread.currentThread().getName();

```

```

        System.out.println "[" + threadName + "] Preparing flight to " +
destination);
        try {
            Thread.sleep(400);
            System.out.println "[" + threadName + "] Flight plan for " +
                boeing777.getModel() + " approved";
            Thread.sleep(400);
            System.out.println "[" + threadName + "] ✓ Ready for departure
to " + destination);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }, "Lambda-Thread-2");

    lambdaThread1.start();
    lambdaThread2.start();

    try {
        lambdaThread1.join();
        lambdaThread2.join();
        System.out.println("\n✓ Все лямбда-задачи завершены");
    } catch (InterruptedException e) {
        System.err.println("Main thread interrupted");
        Thread.currentThread().interrupt();
    }

    //
=====
    // ПРИМЕР 5: Комплексная подготовка к полету
    //
=====
    System.out.println("\n" + "=".repeat(80));
    System.out.println("ПРИМЕР 5: Комплексная подготовка к полету");
    System.out.println("=".repeat(80));
    System.out.println("Демонстрация: параллельное выполнение всех задач
подготовки\n");

    // Создаем все задачи для одного самолета
    RefuelingTask refueling = new RefuelingTask(boeing737, 6000, "Station-
A1");
    BoardingTask boarding = new BoardingTask(boeing737, 189, "Gate-B5");
    PreFlightCheckTask preCheck = new PreFlightCheckTask(boeing737,
"Captain-Smith");

    // Создаем потоки
    Thread refuelThread = new Thread(refueling, "Refuel-B737");
    Thread boardThread = new Thread(boarding, "Board-B737");
    Thread checkThread = new Thread(preCheck, "Check-B737");

    // Запускаем все задачи параллельно

```



```

        System.out.println("Starting all preparation tasks for " +
boeing737.getModel() + "...\\n");
        refuelThread.start();
        boardThread.start();
        checkThread.start();

        // Ждем завершения всех задач
        try {
            refuelThread.join();
            boardThread.join();
            checkThread.join();
            System.out.println("\\n✓ Все задачи подготовки завершены - самолет
готов к вылету!");
        } catch (InterruptedException e) {
            System.err.println("Main thread interrupted");
            Thread.currentThread().interrupt();
        }

        //
=====
        // ПРИМЕР 6: Сравнение Thread vs Runnable
        //
=====

        System.out.println("\\n" + "=".repeat(80));
        System.out.println("ПРИМЕР 6: Сравнение подходов Thread vs Runnable");
        System.out.println("=".repeat(80));

        System.out.println("\\nПодход 1: Наследование от Thread (из Задания 1)");
        PlaneMaintenanceThread threadApproach = new
PlaneMaintenanceThread(airbusA320);
        threadApproach.start();

        System.out.println("\\nПодход 2: Реализация Runnable (Задание 2)");
        RefuelingTask runnableApproach = new RefuelingTask(airbusA320, 4500);
        Thread runnableThread = new Thread(runnableApproach, "Runnable-
Approach");
        runnableThread.start();

        try {
            threadApproach.join();
            runnableThread.join();
            System.out.println("\\n✓ Оба подхода завершены");
        } catch (InterruptedException e) {
            System.err.println("Main thread interrupted");
            Thread.currentThread().interrupt();
        }

        //
=====
        // ИТОГИ

```

```
//
=====

System.out.println("\n" + "=".repeat(80));
System.out.println("ИТОГИ ДЕМОНСТРАЦИИ");
System.out.println("=".repeat(80));
System.out.println("\nПродemonстрированные концепции:");
System.out.println("✓ 1. Реализация интерфейса Runnable");
System.out.println("✓ 2. Создание потоков через new Thread(Runnable)");
System.out.println("✓ 3. Повторное использование одной задачи в разных
потоках");
System.out.println("✓ 4. Анонимные классы для Runnable");
System.out.println("✓ 5. Лямбда-выражения (современный подход)");
System.out.println("✓ 6. Параллельное выполнение множества задач");
System.out.println("✓ 7. Сравнение Thread vs Runnable");

System.out.println("\nГлавный поток (main) завершен!");
System.out.println("=".repeat(80));
}
}
```

3.8 Файл EmergencyLandingTask.java

```
import common.Aeroplane;

public class EmergencyLandingTask implements Runnable {
    private Aeroplane plane;
    private String emergencyType;
    private long startTime;
    private long endTime;

    public EmergencyLandingTask(Aeroplane plane, String emergencyType) {
        this.plane = plane;
        this.emergencyType = emergencyType;
    }

    @Override
    public void run() {
        startTime = System.currentTimeMillis();
        String threadName = Thread.currentThread().getName();
        int priority = Thread.currentThread().getPriority();

        System.out.println "[" + threadName + "] EMERGENCY! Priority: " +
priority);
        System.out.println "[" + threadName + "] " + plane.getModel() + " - " +
emergencyType);

        try {
            System.out.println "[" + threadName + "] ⚠ Declaring
emergency...");
            Thread.sleep(100);

            System.out.println "[" + threadName + "] ⚠ Clearing airspace...");
            Thread.sleep(100);

            System.out.println "[" + threadName + "] ⚠ Preparing emergency
landing...");
            Thread.sleep(150);

            System.out.println "[" + threadName + "] ⚠ Emergency services on
standby...");
            Thread.sleep(100);

            System.out.println "[" + threadName + "] ⚠ Beginning emergency
descent...");
            Thread.sleep(200);

            System.out.println "[" + threadName + "] ⚠ Final approach...");
            Thread.sleep(150);

            System.out.println "[" + threadName + "] ⚠ EMERGENCY LANDING
COMPLETE!");
        }
    }
}
```

```

        endTime = System.currentTimeMillis();
        long duration = endTime - startTime;

        System.out.println "[" + threadName + "] ✓ Emergency handled in " +
duration + "ms");
        System.out.println "[" + threadName + "] Aircraft and passengers
safe!");

    } catch (InterruptedException e) {
        System.out.println "[" + threadName + "] Emergency procedure
interrupted!");
        Thread.currentThread().interrupt();
    }
}

public long getDuration() {
    return endTime - startTime;
}
}

```

3.9 Файл RoutineMaintenanceTask.java

```

import common.Aeroplane;

public class RoutineMaintenanceTask implements Runnable {
    private Aeroplane plane;
    private String maintenanceType;
    private long startTime;
    private long endTime;

    public RoutineMaintenanceTask(Aeroplane plane, String maintenanceType) {
        this.plane = plane;
        this.maintenanceType = maintenanceType;
    }

    @Override
    public void run() {
        startTime = System.currentTimeMillis();
        String threadName = Thread.currentThread().getName();
        int priority = Thread.currentThread().getPriority();

        System.out.println "[" + threadName + "] Starting routine maintenance.
Priority: " + priority);
        System.out.println "[" + threadName + "] " + plane.getModel() + " - " +
maintenanceType);

        try {
            System.out.println "[" + threadName + "] 🔧 Checking systems...";
            Thread.sleep(200);

```

```

        System.out.println "[" + threadName + "] 🔧 Inspecting
components...");
        Thread.sleep(250);

        System.out.println "[" + threadName + "] 🔧 Testing equipment...");
        Thread.sleep(200);

        System.out.println "[" + threadName + "] 🔧 Replacing filters...");
        Thread.sleep(180);

        System.out.println "[" + threadName + "] 🔧 Lubricating parts...");
        Thread.sleep(150);

        System.out.println "[" + threadName + "] 🔧 Final inspection...");
        Thread.sleep(200);

        System.out.println "[" + threadName + "] 🔧 Updating maintenance
log...");
        Thread.sleep(100);

        endTime = System.currentTimeMillis();
        long duration = endTime - startTime;

        System.out.println "[" + threadName + "] ✓ Routine maintenance
completed in " + duration + "ms");

    } catch (InterruptedException e) {
        System.out.println "[" + threadName + "] Maintenance interrupted!";
        Thread.currentThread().interrupt();
    }
}

public long getDuration() {
    return endTime - startTime;
}
}

```

3.10 Файл DataLoggingTask.java

```

import common.Aeroplane;

public class DataLoggingTask implements Runnable {
    private Aeroplane plane;
    private int recordCount;
    private long startTime;
    private long endTime;

    public DataLoggingTask(Aeroplane plane, int recordCount) {
        this.plane = plane;
    }
}

```

```

        this.recordCount = recordCount;
    }

    @Override
    public void run() {
        startTime = System.currentTimeMillis();
        String threadName = Thread.currentThread().getName();
        int priority = Thread.currentThread().getPriority();

        System.out.println "[" + threadName + "] Starting background logging.
Priority: " + priority);
        System.out.println "[" + threadName + "] " + plane.getModel() + " -
Logging " + recordCount + " records");

        try {
            for (int i = 1; i <= recordCount; i++) {
                System.out.println "[" + threadName + "] 📝 Logging record " + i
+ "/" + recordCount +
                    " for " + plane.getModel());

                Thread.sleep(200);

                if (i % 3 == 0) {
                    System.out.println "[" + threadName + "] 📝 Compressing
data...");
                    Thread.sleep(100);
                }
            }

            System.out.println "[" + threadName + "] 📝 Finalizing log
file...");
            Thread.sleep(150);

            endTime = System.currentTimeMillis();
            long duration = endTime - startTime;

            System.out.println "[" + threadName + "] ✓ Logging completed in " +
duration + "ms");
            System.out.println "[" + threadName + "] Total records: " +
recordCount);

        } catch (InterruptedException e) {
            System.out.println "[" + threadName + "] Logging interrupted!";
            Thread.currentThread().interrupt();
        }
    }

    public long getDuration() {
        return endTime - startTime;
    }
}

```

3.11 Файл PriorityDemo.java

```
import common.Aeroplane;
import common.Engine;

/**
 * Демонстрация работы с приоритетами потоков
 *
 * Задание 3: Назначение приоритетов потокам
 *
 * Ключевые концепции:
 * 1. Установка приоритетов через setPriority()
 * 2. Использование констант MIN_PRIORITY, NORM_PRIORITY, MAX_PRIORITY
 * 3. Влияние приоритетов на порядок выполнения
 * 4. Практические сценарии использования приоритетов
 */
public class PriorityDemo {

    public static void main(String[] args) {
        System.out.println("=".repeat(80));
        System.out.println("ЗАДАНИЕ 3: ДЕМОНСТРАЦИЯ РАБОТЫ С ПРИОРИТЕТАМИ ПОТОКОВ");
        System.out.println("=".repeat(80));
        System.out.println();

        // Информация о приоритетах
        System.out.println("Диапазон приоритетов в Java:");
        System.out.println("  MIN_PRIORITY = " + Thread.MIN_PRIORITY + " (минимальный)");
        System.out.println("  NORM_PRIORITY = " + Thread.NORM_PRIORITY + " (по умолчанию)");
        System.out.println("  MAX_PRIORITY = " + Thread.MAX_PRIORITY + " (максимальный)");
        System.out.println();

        // Создаем самолеты
        Aeroplane boeing737 = new Aeroplane(
            "Boeing", "737-800", 2015, 850.0,
            189, 35.8, new Engine("CFM56-7B", 27300)
        );

        Aeroplane airbusA320 = new Aeroplane(
            "Airbus", "A320", 2018, 840.0,
            180, 35.8, new Engine("CFM56-5B", 27000)
        );

        Aeroplane boeing777 = new Aeroplane(
            "Boeing", "777-300ER", 2020, 905.0,
            396, 64.8, new Engine("GE90-115B", 52000)
        );
    }
}
```

```

);

//
=====
// ПРИМЕР 1: Базовая установка приоритетов
//
=====
System.out.println("\n" + "=".repeat(80));
System.out.println("ПРИМЕР 1: Базовая установка приоритетов");
System.out.println("=".repeat(80));
System.out.println("Демонстрация: создание потоков с разными
приоритетами\n");

// Создаем задачи
EmergencyLandingTask emergency = new EmergencyLandingTask(boeing737,
"Engine failure");
RoutineMaintenanceTask maintenance = new
RoutineMaintenanceTask(airbusA320, "Scheduled check");
DataLoggingTask logging = new DataLoggingTask(boeing777, 5);

// Создаем потоки
Thread emergencyThread = new Thread(emergency, "Emergency-Thread");
Thread maintenanceThread = new Thread(maintenance, "Maintenance-
Thread");
Thread loggingThread = new Thread(logging, "Logging-Thread");

// Устанавливаем приоритеты
emergencyThread.setPriority(Thread.MAX_PRIORITY); // 10 - критическая
задача
maintenanceThread.setPriority(Thread.NORM_PRIORITY); // 5 - обычная
задача
loggingThread.setPriority(Thread.MIN_PRIORITY); // 1 - фоновая
задача

System.out.println("Приоритеты установлены:");
System.out.println(" " + emergencyThread.getName() + ": " +
emergencyThread.getPriority());
System.out.println(" " + maintenanceThread.getName() + ": " +
maintenanceThread.getPriority());
System.out.println(" " + loggingThread.getName() + ": " +
loggingThread.getPriority());
System.out.println();

// Запускаем потоки
emergencyThread.start();
maintenanceThread.start();
loggingThread.start();

try {
    emergencyThread.join();
    maintenanceThread.join();

```



```

        loggingThread.join();
        System.out.println("\n✓ Все задачи завершены");
    } catch (InterruptedException e) {
        System.err.println("Main thread interrupted");
        Thread.currentThread().interrupt();
    }

    //
=====
    // ПРИМЕР 2: Сравнение времени выполнения
    //
=====
    System.out.println("\n" + "=".repeat(80));
    System.out.println("ПРИМЕР 2: Влияние приоритетов на время выполнения");
    System.out.println("=".repeat(80));
    System.out.println("Демонстрация: запуск одинаковых задач с разными
приоритетами\n");

    // Создаем три одинаковые задачи
    DataLoggingTask highPriorityTask = new DataLoggingTask(boeing737, 8);
    DataLoggingTask normalPriorityTask = new DataLoggingTask(airbusA320, 8);
    DataLoggingTask lowPriorityTask = new DataLoggingTask(boeing777, 8);

    Thread highThread = new Thread(highPriorityTask, "High-Priority");
    Thread normalThread = new Thread(normalPriorityTask, "Normal-Priority");
    Thread lowThread = new Thread(lowPriorityTask, "Low-Priority");

    highThread.setPriority(Thread.MAX_PRIORITY);
    normalThread.setPriority(Thread.NORM_PRIORITY);
    lowThread.setPriority(Thread.MIN_PRIORITY);

    System.out.println("Запуск трех одинаковых задач с разными
приоритетами...\n");

    // Запускаем одновременно
    highThread.start();
    normalThread.start();
    lowThread.start();

    try {
        highThread.join();
        normalThread.join();
        lowThread.join();

        System.out.println("\n--- Результаты ---");
        System.out.println("High Priority (10):  " +
highPriorityTask.getDuration() + "ms");
        System.out.println("Normal Priority (5):  " +
normalPriorityTask.getDuration() + "ms");
        System.out.println("Low Priority (1):    " +
lowPriorityTask.getDuration() + "ms");
    }

```

```

        System.out.println("\nПримечание: Разница может быть незначительной
из-за особенностей планировщика ОС");
    } catch (InterruptedException e) {
        System.err.println("Main thread interrupted");
        Thread.currentThread().interrupt();
    }

    //
=====
    // ПРИМЕР 3: Изменение приоритета во время выполнения
    //
=====

    System.out.println("\n" + "=".repeat(80));
    System.out.println("ПРИМЕР 3: Динамическое изменение приоритета");
    System.out.println("=".repeat(80));
    System.out.println("Демонстрация: изменение приоритета работающего
потока\n");

    DataLoggingTask dynamicTask = new DataLoggingTask(boeing737, 6);
    Thread dynamicThread = new Thread(dynamicTask, "Dynamic-Priority");

    System.out.println("Начальный приоритет: " +
dynamicThread.getPriority());
    dynamicThread.setPriority(Thread.MIN_PRIORITY);
    System.out.println("Установлен MIN_PRIORITY: " +
dynamicThread.getPriority());

    dynamicThread.start();

    try {
        Thread.sleep(1000); // Даем потоку поработать

        System.out.println("\n>>> Повышаем приоритет до MAX_PRIORITY");
        dynamicThread.setPriority(Thread.MAX_PRIORITY);
        System.out.println("Новый приоритет: " + dynamicThread.getPriority()
+ "\n");

        dynamicThread.join();
        System.out.println("\n✓ Задача завершена");
    } catch (InterruptedException e) {
        System.err.println("Main thread interrupted");
        Thread.currentThread().interrupt();
    }

    //
=====
    // ПРИМЕР 4: Пользовательские приоритеты
    //
=====

    System.out.println("\n" + "=".repeat(80));

```

```

        System.out.println("ПРИМЕР 4: Использование пользовательских значений
приоритетов");
        System.out.println("=".repeat(80));
        System.out.println("Демонстрация: установка произвольных значений от 1
до 10\n");

        // Создаем потоки с разными приоритетами
        Thread[] threads = new Thread[5];
        int[] priorities = {1, 3, 5, 7, 10};

        for (int i = 0; i < 5; i++) {
            final int taskNum = i + 1;
            threads[i] = new Thread(() -> {
                String name = Thread.currentThread().getName();
                int priority = Thread.currentThread().getPriority();
                System.out.println "[" + name + "] Running with priority " +
priority);

                try {
                    Thread.sleep(300);
                    System.out.println "[" + name + "] Completed";
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }, "Task-" + taskNum);

            threads[i].setPriority(priorities[i]);
            System.out.println("Created " + threads[i].getName() + " with
priority " + priorities[i]);
        }

        System.out.println("\nЗапуск всех потоков...\n");

        for (Thread thread : threads) {
            thread.start();
        }

        try {
            for (Thread thread : threads) {
                thread.join();
            }
            System.out.println("\n✓ Все задачи завершены");
        } catch (InterruptedException e) {
            System.err.println("Main thread interrupted");
            Thread.currentThread().interrupt();
        }

        //
=====
        // ПРИМЕР 5: Практический сценарий - Управление воздушным движением
        //
=====

```

```

System.out.println("\n" + "=".repeat(80));
System.out.println("ПРИМЕР 5: Практический сценарий - Управление
воздушным движением");
System.out.println("=".repeat(80));
System.out.println("Демонстрация: реалистичное использование
приоритетов\n");

// Критическая ситуация
EmergencyLandingTask emergency1 = new EmergencyLandingTask(
    boeing737, "Medical emergency"
);

// Обычные операции
RoutineMaintenanceTask routine1 = new RoutineMaintenanceTask(
    airbusA320, "Pre-flight inspection"
);

RefuelingTask refuel = new RefuelingTask(boeing777, 8000);

// Фоновые задачи
DataLoggingTask log1 = new DataLoggingTask(boeing737, 4);
DataLoggingTask log2 = new DataLoggingTask(airbusA320, 4);

// Создаем потоки
Thread emergThread = new Thread(emergency1, "EMERGENCY");
Thread routineThread = new Thread(routine1, "Routine-Ops");
Thread refuelThread = new Thread(refuel, "Refuel-Ops");
Thread log1Thread = new Thread(log1, "Background-Log-1");
Thread log2Thread = new Thread(log2, "Background-Log-2");

// Устанавливаем приоритеты согласно важности
emergThread.setPriority(Thread.MAX_PRIORITY); // 10 - критично!
routineThread.setPriority(Thread.NORM_PRIORITY); // 5 - обычная работа
refuelThread.setPriority(Thread.NORM_PRIORITY); // 5 - обычная работа
log1Thread.setPriority(Thread.MIN_PRIORITY); // 1 - фон
log2Thread.setPriority(Thread.MIN_PRIORITY); // 1 - фон

System.out.println("Приоритеты операций:");
System.out.println(" 🚨 " + emergThread.getName() + ": " +
emergThread.getPriority() + " (КРИТИЧНО)");
System.out.println(" 🔧 " + routineThread.getName() + ": " +
routineThread.getPriority() + " (обычная)");
System.out.println(" ⛽ " + refuelThread.getName() + ": " +
refuelThread.getPriority() + " (обычная)");
System.out.println(" 📝 " + log1Thread.getName() + ": " +
log1Thread.getPriority() + " (фон)");
System.out.println(" 📝 " + log2Thread.getName() + ": " +
log2Thread.getPriority() + " (фон)");
System.out.println();

// Запускаем все операции

```

```

emergThread.start();
routineThread.start();
refuelThread.start();
log1Thread.start();
log2Thread.start();

try {
    emergThread.join();
    routineThread.join();
    refuelThread.join();
    log1Thread.join();
    log2Thread.join();
    System.out.println("\n✓ Все операции завершены");
} catch (InterruptedException e) {
    System.err.println("Main thread interrupted");
    Thread.currentThread().interrupt();
}

//
=====
// ИТОГИ
//
=====

System.out.println("\n" + "=".repeat(80));
System.out.println("ИТОГИ ДЕМОНСТРАЦИИ");
System.out.println("=".repeat(80));
System.out.println("\nПродemonстрированные концепции:");
System.out.println("✓ 1. Установка приоритетов через setPriority()");
System.out.println("✓ 2. Использование констант MIN/NORM/MAX_PRIORITY");
System.out.println("✓ 3. Пользовательские значения приоритетов (1-10)");
System.out.println("✓ 4. Динамическое изменение приоритета");
System.out.println("✓ 5. Влияние приоритетов на выполнение");
System.out.println("✓ 6. Практические сценарии использования");

System.out.println("\nГлавный поток (main) завершен!");
System.out.println("=".repeat(80));
}
}

```

3.12 Файл RunwayController.java

```

public class RunwayController {
    private String runwayName;
    private boolean isOccupied;
    private String currentUser;

    public RunwayController(String runwayName) {
        this.runwayName = runwayName;
        this.isOccupied = false;
        this.currentUser = null;
    }
}

```

```

    }

    public synchronized void requestRunway(String aircraftId, String operation)
    {
        String threadName = Thread.currentThread().getName();

        System.out.println "[" + threadName + "] " + aircraftId +
            " requesting " + runwayName + " for " + operation);

        while (isOccupied) {
            try {
                System.out.println "[" + threadName + "] " + aircraftId +
                    " waiting... (occupied by " + currentUser +
" )");
                wait();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }

        isOccupied = true;
        currentUser = aircraftId;
        System.out.println "[" + threadName + "] ✓ " + aircraftId +
            " acquired " + runwayName + " for " + operation);
    }

    public synchronized void useRunway(String aircraftId, String operation) {
        String threadName = Thread.currentThread().getName();

        try {
            System.out.println "[" + threadName + "] >>> " + aircraftId +
                " is " + operation + " on " + runwayName);

            if (operation.contains("taking off")) {
                Thread.sleep(1500);
            } else if (operation.contains("landing")) {
                Thread.sleep(1200);
            } else {
                Thread.sleep(1000);
            }

            System.out.println "[" + threadName + "] <<< " + aircraftId +
                " completed " + operation);

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    public synchronized void releaseRunway(String aircraftId) {

```

```

String threadName = Thread.currentThread().getName();

if (currentUser != null && currentUser.equals(aircraftId)) {
    isOccupied = false;
    currentUser = null;
    System.out.println "[" + threadName + "] ✓ " + aircraftId +
        " released " + runwayName);
    notifyAll();
}
}

public void performOperation(String aircraftId, String operation) {
    requestRunway(aircraftId, operation);
    useRunway(aircraftId, operation);
    releaseRunway(aircraftId);
}

public String getRunwayName() {
    return runwayName;
}

public synchronized boolean isAvailable() {
    return !isOccupied;
}
}

```

3.13 Файл FuelTank.java

```

public class FuelTank {
    private double capacity;           // Максимальная вместимость
    private double currentLevel;       // Текущий уровень топлива
    private String tankId;

    public FuelTank(String tankId, double capacity) {
        this.tankId = tankId;
        this.capacity = capacity;
        this.currentLevel = 0;
    }

    public synchronized void refill(double amount, String supplier) {
        String threadName = Thread.currentThread().getName();

        while (currentLevel + amount > capacity) {
            try {
                System.out.println "[" + threadName + "] " + supplier +
                    " waiting... Tank " + tankId + " is full (" +
                    String.format("%.0f", currentLevel) + "/" +
                    String.format("%.0f", capacity) + " liters)";

                wait();
            } catch (InterruptedException e) {

```

```

        Thread.currentThread().interrupt();
        return;
    }
}

currentLevel += amount;
System.out.println "[" + threadName + "] 🛢️ " + supplier +
    " refilled Tank " + tankId + " with " +
    String.format("%.0f", amount) + " liters");
System.out.println "[" + threadName + "] Tank " + tankId + " level: " +
    String.format("%.0f", currentLevel) + "/" +
    String.format("%.0f", capacity) + " liters");

notifyAll();
}

public synchronized void withdraw(double amount, String aircraft) {
    String threadName = Thread.currentThread().getName();

    while (currentLevel < amount) {
        try {
            System.out.println "[" + threadName + "] " + aircraft +
                " waiting for fuel... Tank " + tankId +
                " has only " + String.format("%.0f",
currentLevel) +
                " liters (need " + String.format("%.0f",
amount) + ")");
            wait();
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return;
        }
    }

    currentLevel -= amount;
    System.out.println "[" + threadName + "] ✂️ " + aircraft +
        " withdrew " + String.format("%.0f", amount) +
        " liters from Tank " + tankId);
    System.out.println "[" + threadName + "] Tank " + tankId + " level: " +
        String.format("%.0f", currentLevel) + "/" +
        String.format("%.0f", capacity) + " liters");

    notifyAll();
}

public synchronized double getCurrentLevel() {
    return currentLevel;
}

public synchronized double getAvailableSpace() {
    return capacity - currentLevel;
}

```



```

    }

    public String getTankId() {
        return tankId;
    }

    public double getCapacity() {
        return capacity;
    }
}

```

3.14 Файл UnsafeCounter.java

```

public class UnsafeCounter {
    private int count = 0;

    public void increment() {
        count++;
    }

    public int getCount() {
        return count;
    }
}

class SafeCounter {
    private int count = 0;

    public synchronized void increment() {
        count++;
    }

    public synchronized int getCount() {
        return count;
    }
}

```

3.15 Файл SynchronizationDemo.java

```

import common.Aeroplane;
import common.Engine;

/**
 * Демонстрация синхронизации потоков
 *
 * Задание 4: Синхронизация потоков выполнения
 *
 * Ключевые концепции:
 * 1. Проблема race condition
 * 2. Ключевое слово synchronized

```

```

* 3. Методы wait() и notify()
* 4. Паттерн Producer-Consumer
* 5. Взаимное исключение (mutual exclusion)
*/
public class SynchronizationDemo {

    public static void main(String[] args) {
        System.out.println("=".repeat(80));
        System.out.println("ЗАДАНИЕ 4: ДЕМОНСТРАЦИЯ СИНХРОНИЗАЦИИ ПОТОКОВ");
        System.out.println("=".repeat(80));
        System.out.println();

        //
=====
        // ПРИМЕР 1: Проблема Race Condition
        //
=====
        System.out.println("=".repeat(80));
        System.out.println("ПРИМЕР 1: Демонстрация проблемы Race Condition");
        System.out.println("=".repeat(80));
        System.out.println("Проблема: несколько потоков изменяют общий ресурс
без синхронизации\n");

        demonstrateRaceCondition();

        //
=====
        // ПРИМЕР 2: Решение через synchronized
        //
=====
        System.out.println("\n" + "=".repeat(80));
        System.out.println("ПРИМЕР 2: Решение проблемы через synchronized");
        System.out.println("=".repeat(80));
        System.out.println("Решение: использование synchronized для защиты
критической секции\n");

        demonstrateSynchronizedSolution();

        //
=====
        // ПРИМЕР 3: Синхронизация доступа к взлетно-посадочной полосе
        //
=====
        System.out.println("\n" + "=".repeat(80));
        System.out.println("ПРИМЕР 3: Синхронизация доступа к общему ресурсу");
        System.out.println("=".repeat(80));
        System.out.println("Сценарий: несколько самолетов используют одну
взлетно-посадочную полосу\n");

        demonstrateRunwayControl();
    }
}

```

```

//
=====
// ПРИМЕР 4: Паттерн Producer-Consumer с wait/notify
//
=====
System.out.println("\n" + "=".repeat(80));
System.out.println("ПРИМЕР 4: Паттерн Producer-Consumer");
System.out.println("=".repeat(80));
System.out.println("Сценарий: поставщики заправляют резервуар, самолеты
забирают топливо\n");

    demonstrateProducerConsumer();

//
=====
// ПРИМЕР 5: Комплексная синхронизация
//
=====
System.out.println("\n" + "=".repeat(80));
System.out.println("ПРИМЕР 5: Комплексная синхронизация операций
аэропорта");
System.out.println("=".repeat(80));
System.out.println("Сценарий: координация взлетов, посадок и
заправки\n");

    demonstrateComplexSynchronization();

//
=====
// ИТОГИ
//
=====
System.out.println("\n" + "=".repeat(80));
System.out.println("ИТОГИ ДЕМОНСТРАЦИИ");
System.out.println("=".repeat(80));
System.out.println("\nПродемонстрированные концепции:");
System.out.println("✓ 1. Проблема race condition и ее последствия");
System.out.println("✓ 2. Использование synchronized для защиты
критических секций");
System.out.println("✓ 3. Методы wait() и notify() для координации
потоков");
System.out.println("✓ 4. Паттерн Producer-Consumer");
System.out.println("✓ 5. Взаимное исключение при доступе к общим
ресурсам");
System.out.println("✓ 6. Практические сценарии синхронизации");

System.out.println("\nГлавный поток (main) завершен!");
System.out.println("=".repeat(80));
}

/**

```

```

* Демонстрация проблемы race condition
*/
private static void demonstrateRaceCondition() {
    final int ITERATIONS = 1000;
    final int THREAD_COUNT = 10;

    UnsafeCounter unsafeCounter = new UnsafeCounter();

    System.out.println("Запуск " + THREAD_COUNT + " потоков, каждый
увеличивает счетчик " +
        ITERATIONS + " раз");
    System.out.println("Ожидаемый результат: " + (THREAD_COUNT *
ITERATIONS));

    Thread[] threads = new Thread[THREAD_COUNT];

    for (int i = 0; i < THREAD_COUNT; i++) {
        threads[i] = new Thread(() -> {
            for (int j = 0; j < ITERATIONS; j++) {
                unsafeCounter.increment();
            }
        }, "UnsafeThread-" + (i + 1));
        threads[i].start();
    }

    // Ждем завершения всех потоков
    try {
        for (Thread thread : threads) {
            thread.join();
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }

    System.out.println("Фактический результат: " +
unsafeCounter.getCount());
    System.out.println("✗ Результат НЕВЕРНЫЙ из-за race condition!");
    System.out.println("    Потоки одновременно изменяли счетчик без
синхронизации");
}

/**
* Демонстрация решения через synchronized
*/
private static void demonstrateSynchronizedSolution() {
    final int ITERATIONS = 1000;
    final int THREAD_COUNT = 10;

    SafeCounter safeCounter = new SafeCounter();

```

```

        System.out.println("Запуск " + THREAD_COUNT + " потоков, каждый
увеличивает счетчик " +
            ITERATIONS + " раз");
        System.out.println("Ожидаемый результат: " + (THREAD_COUNT *
ITERATIONS));

        Thread[] threads = new Thread[THREAD_COUNT];

        for (int i = 0; i < THREAD_COUNT; i++) {
            threads[i] = new Thread(() -> {
                for (int j = 0; j < ITERATIONS; j++) {
                    safeCounter.increment();
                }
            }, "SafeThread-" + (i + 1));
            threads[i].start();
        }

        // Ждем завершения всех потоков
        try {
            for (Thread thread : threads) {
                thread.join();
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

        System.out.println("Фактический результат: " + safeCounter.getCount());
        System.out.println("✓ Результат ВЕРНЫЙ благодаря synchronized!");
        System.out.println("Синхронизация гарантирует, что только один поток
изменяет счетчик");
    }

    /**
     * Демонстрация контроля доступа к взлетно-посадочной полосе
     */
    private static void demonstrateRunwayControl() {
        RunwayController runway = new RunwayController("Runway-01");

        // Создаем самолеты
        String[] aircraft = {"Boeing-737", "Airbus-A320", "Boeing-777", "Airbus-
A380"};
        String[] operations = {"taking off", "landing", "taking off",
"landing"};

        Thread[] threads = new Thread[aircraft.length];

        for (int i = 0; i < aircraft.length; i++) {
            final String plane = aircraft[i];
            final String operation = operations[i];

            threads[i] = new Thread(() -> {

```

```

        runway.performOperation(plane, operation);
    }, plane + "-Thread");

    threads[i].start();
}

// Ждем завершения всех операций
try {
    for (Thread thread : threads) {
        thread.join();
    }
    System.out.println("\n✓ Все операции на полосе завершены
безопасно");
    System.out.println("    Синхронизация предотвратила одновременное
использование полосы");
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}

}

/**
 * Демонстрация паттерна Producer-Consumer
 */
private static void demonstrateProducerConsumer() {
    FuelTank tank = new FuelTank("Main-Tank", 10000);

    // Производители (поставщики топлива)
    Thread supplier1 = new Thread(() -> {
        try {
            for (int i = 1; i <= 3; i++) {
                Thread.sleep(800);
                tank.refill(3000, "Supplier-1");
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }, "Supplier-1-Thread");

    Thread supplier2 = new Thread(() -> {
        try {
            for (int i = 1; i <= 2; i++) {
                Thread.sleep(1200);
                tank.refill(4000, "Supplier-2");
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }, "Supplier-2-Thread");

    // Потребители (самолеты)
    Thread aircraft1 = new Thread(() -> {

```

```

        try {
            Thread.sleep(500);
            tank.withdraw(2500, "Boeing-737");
            Thread.sleep(1000);
            tank.withdraw(2500, "Boeing-737");
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }, "Boeing-737-Thread");

    Thread aircraft2 = new Thread(() -> {
        try {
            Thread.sleep(700);
            tank.withdraw(3500, "Airbus-A320");
            Thread.sleep(1200);
            tank.withdraw(3000, "Airbus-A320");
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }, "Airbus-A320-Thread");

    // Запускаем всех
    supplier1.start();
    supplier2.start();
    aircraft1.start();
    aircraft2.start();

    try {
        supplier1.join();
        supplier2.join();
        aircraft1.join();
        aircraft2.join();

        System.out.println("\n✓ Все операции с топливом завершены");
        System.out.println("    Finalный уровень в резервуаре: " +
            String.format("%.0f", tank.getCurrentLevel()) + "
литров");

        System.out.println("    wait/notify обеспечили корректную
координацию");
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}

/**
 * Комплексная демонстрация синхронизации
 */
private static void demonstrateComplexSynchronization() {
    RunwayController runway = new RunwayController("Runway-02");
    FuelTank tank = new FuelTank("Airport-Tank", 15000);

```

```

// Заправляем резервуар достаточным количеством топлива
Thread supplier = new Thread(() -> {
    try {
        tank.refill(13000, "Initial-Supply");
    } catch (Exception e) {
        e.printStackTrace();
    }
}, "Supplier-Thread");
supplier.start();

try {
    supplier.join();
    Thread.sleep(500);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}

// Создаем самолеты с полным циклом операций
Thread flight1 = new Thread(() -> {
    try {
        // Заправка
        tank.withdraw(4000, "Flight-AA101");
        Thread.sleep(300);
        // Взлет
        runway.performOperation("Flight-AA101", "taking off");
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}, "Flight-AA101");

Thread flight2 = new Thread(() -> {
    try {
        // Посадка
        runway.performOperation("Flight-BA202", "landing");
        Thread.sleep(300);
        // Заправка
        tank.withdraw(3500, "Flight-BA202");
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}, "Flight-BA202");

Thread flight3 = new Thread(() -> {
    try {
        Thread.sleep(200);
        // Заправка
        tank.withdraw(4500, "Flight-LH303");
        Thread.sleep(300);
        // Взлет
        runway.performOperation("Flight-LH303", "taking off");
    } catch (InterruptedException e) {

```



```

        Thread.currentThread().interrupt();
    }
}, "Flight-LH303");

System.out.println("Запуск комплексных операций...\n");

flight1.start();
flight2.start();
flight3.start();

try {
    flight1.join();
    flight2.join();
    flight3.join();

    System.out.println("\n✓ Все комплексные операции завершены
успешно");
    System.out.println("    Синхронизация обеспечила безопасную
координацию:");
    System.out.println("    - Взлетно-посадочная полоса использовалась по
очереди");
    System.out.println("    - Топливо распределялось корректно");
    System.out.println("    - Не было конфликтов между потоками");
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
}
}

```

3.16 Файл App.java

```

import java.util.Scanner;

public class App {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("=".repeat(80));
        System.out.println("ЛАБОРАТОРНАЯ РАБОТА №3: МНОГОПОТОЧНОСТЬ В JAVA");
        System.out.println("=".repeat(80));

        waitForUser(scanner, "\nНажмите Enter для запуска Задания 1
(Thread) ...");
        runDemo1();

        waitForUser(scanner, "\nНажмите Enter для запуска Задания 2
(Runnable) ...");
        runDemo2();
    }
}

```

```

        waitForUser(scanner, "\nНажмите Enter для запуска Задания 3
(Приоритеты) ...");
        runDemo3();

        waitForUser(scanner, "\nНажмите Enter для запуска Задания 4
(Синхронизация) ...");
        runDemo4();

        scanner.close();
    }

    private static void waitForUser(Scanner scanner, String message) {
        System.out.println(message);
        scanner.nextLine();
    }

    private static void runDemo1() {
        System.out.println("\n" + "=".repeat(80));
        System.out.println("ЗАПУСК ЗАДАНИЯ 1: КЛАСС THREAD");
        System.out.println("=".repeat(80));
        System.out.println();

        try {
            ThreadDemo.main(new String[]{});
        } catch (Exception e) {
            System.err.println("Ошибка при выполнении ThreadDemo: " +
e.getMessage());
            e.printStackTrace();
        }
    }

    private static void runDemo2() {
        System.out.println("\n" + "=".repeat(80));
        System.out.println("ЗАПУСК ЗАДАНИЯ 2: ИНТЕРФЕЙС RUNNABLE");
        System.out.println("=".repeat(80));
        System.out.println();

        try {
            RunnableDemo.main(new String[]{});
        } catch (Exception e) {
            System.err.println("Ошибка при выполнении RunnableDemo: " +
e.getMessage());
            e.printStackTrace();
        }
    }

    private static void runDemo3() {
        System.out.println("\n" + "=".repeat(80));
        System.out.println("ЗАПУСК ЗАДАНИЯ 3: ПРИОРИТЕТЫ ПОТОКОВ");
        System.out.println("=".repeat(80));
        System.out.println();
    }

```

```

        try {
            PriorityDemo.main(new String[]{});
        } catch (Exception e) {
            System.err.println("Ошибка при выполнении PriorityDemo: " +
e.getMessage());
            e.printStackTrace();
        }
    }

    private static void runDemo4() {
        System.out.println("\n" + "=".repeat(80));
        System.out.println("ЗАПУСК ЗАДАНИЯ 4: СИНХРОНИЗАЦИЯ ПОТОКОВ");
        System.out.println("=".repeat(80));
        System.out.println();

        try {
            SynchronizationDemo.main(new String[]{});
        } catch (Exception e) {
            System.err.println("Ошибка при выполнении SynchronizationDemo: " +
e.getMessage());
            e.printStackTrace();
        }
    }
}

```

4 Результат работы программы

```
=====
ЛАБОРАТОРНАЯ РАБОТА №3: МНОГОПОТОЧНОСТЬ В JAVA
=====
```

Нажмите Enter для запуска Задания 1 (Thread)...

```
=====
ЗАПУСК ЗАДАНИЯ 1: КЛАСС THREAD
=====
```

```
=====
ЗАДАНИЕ 1: ДЕМОНСТРАЦИЯ РАБОТЫ С КЛАССОМ THREAD
=====
```

Создано 3 самолета для демонстрации параллельной работы потоков

```
=====
ПРИМЕР 1: Базовое создание и запуск потоков
=====
```

Демонстрация: создание потоков через наследование от Thread

Запускаем 2 потока обслуживания параллельно

```
[Maintenance-A320] Starting maintenance for A320
[Maintenance-737-800] Starting maintenance for 737-800
[Maintenance-A320] Checking engine systems...
[Maintenance-737-800] Checking engine systems...
[Maintenance-737-800] Testing avionics...
[Maintenance-A320] Testing avionics...
[Maintenance-A320] Inspecting airframe...
[Maintenance-737-800] Inspecting airframe...
[Maintenance-A320] Checking fuel systems...
[Maintenance-737-800] Checking fuel systems...
[Maintenance-A320] Performing final checks...
[Maintenance-737-800] Performing final checks...
```

*** Performing maintenance on A320 ***

Checking engine...

Inspecting wings and fuselage...

Testing avionics...

Maintenance complete!

[Maintenance-A320] Maintenance completed for A320

*** Performing maintenance on 737-800 ***

```
Checking engine...
Inspecting wings and fuselage...
Testing avionics...
Maintenance complete!
[Maintenance-737-800] Maintenance completed for 737-800
```

✓ Оба потока обслуживания завершены

```
=====
ПРИМЕР 2: Использование пользовательских имен потоков
=====
```

Демонстрация: именование потоков для лучшей идентификации

```
[CustomMaintenance-B777] Starting maintenance for 777-300ER
[CustomMaintenance-B777] Checking engine systems...
[CustomMaintenance-B777] Testing avionics...
[CustomMaintenance-B777] Inspecting airframe...
[CustomMaintenance-B777] Checking fuel systems...
[CustomMaintenance-B777] Performing final checks...
```

*** Performing maintenance on 777-300ER ***

```
Checking engine...
Inspecting wings and fuselage...
Testing avionics...
Maintenance complete!
[CustomMaintenance-B777] Maintenance completed for 777-300ER
```

✓ Поток с пользовательским именем завершен

```
=====
ПРИМЕР 3: Параллельное выполнение разных типов потоков
=====
```

Демонстрация: одновременное обслуживание и полет разных самолетов

```
[Flight-737-800-to-Moscow] Starting flight simulation for 737-800
[Flight-737-800-to-Moscow] Pre-flight checks...
[Maintenance-A320] Starting maintenance for A320
[Maintenance-A320] Checking engine systems...
[Flight-777-300ER-to-London] Starting flight simulation for 777-300ER
[Flight-777-300ER-to-London] Pre-flight checks...
[Flight-737-800-to-Moscow] Starting engines...
[Flight-777-300ER-to-London] Starting engines...
```

=== Starting Boeing 737-800 ===

=== Starting Boeing 777-300ER ===

Engine GE90-115B starting...

Engine CFM56-7B starting...
 Aircraft systems initialized.
 Aircraft systems initialized.
 [Flight-737-800-to-Moscow] Taking off to Moscow

 >>> 737-800 is taking off...
 Engine CFM56-7B quick starting!
 [Flight-777-300ER-to-London] Taking off to London

 >>> 777-300ER is taking off...
 Engine GE90-115B quick starting!
 Accelerating on runway...
 Lifting off!
 Accelerating on runway...
 Lifting off!
 [Maintenance-A320] Testing avionics...
 [Maintenance-A320] Inspecting airframe...
 [Flight-737-800-to-Moscow] Climbing to cruise altitude...
 Flying at 10000 meters altitude.
 [Flight-777-300ER-to-London] Climbing to cruise altitude...
 Flying at 10000 meters altitude.
 [Maintenance-A320] Checking fuel systems...
 [Flight-737-800-to-Moscow] Cruising at 10,000m to Moscow
 [Flight-777-300ER-to-London] Cruising at 10,000m to London
 [Flight-737-800-to-Moscow] Flight hour 1 completed
 [Flight-777-300ER-to-London] Flight hour 1 completed
 [Maintenance-A320] Performing final checks...

 *** Performing maintenance on A320 ***
 Checking engine...
 Inspecting wings and fuselage...
 Testing avionics...
 Maintenance complete!
 [Maintenance-A320] Maintenance completed for A320
 [Flight-737-800-to-Moscow] Flight hour 2 completed
 [Flight-777-300ER-to-London] Flight hour 2 completed
 [Flight-737-800-to-Moscow] Flight hour 3 completed
 [Flight-777-300ER-to-London] Flight hour 3 completed
 [Flight-737-800-to-Moscow] Descending for landing...
 [Flight-777-300ER-to-London] Descending for landing...
 [Flight-737-800-to-Moscow] Landing at Moscow

 <<< 737-800 is landing...
 Descending...
 Touchdown!
 Taxiing to gate.
 [Flight-777-300ER-to-London] Landing at London

```

<<< 777-300ER is landing...
Descending...
Touchdown!
Taxiing to gate.
[Flight-737-800-to-Moscow] Shutting down engines...

=== Stopping Boeing 737-800 ===
[Flight-777-300ER-to-London] Shutting down engines...

=== Stopping Boeing 777-300ER ===
Engine GE90-115B stopped.
Aircraft systems shut down.
[Flight-777-300ER-to-London] Flight completed! Total hours: 5
Engine CFM56-7B stopped.
Aircraft systems shut down.
[Flight-737-800-to-Moscow] Flight completed! Total hours: 5

```

✓ Все потоки завершены

```

=====
ПРИМЕР 4: Получение информации о потоках
=====

```

Информация о потоке ДО запуска:

- Имя: Maintenance-737-800
- ID: 31
- Состояние: NEW
- Приоритет: 5
- Жив: false

Информация о потоке ПОСЛЕ запуска:

- Состояние: RUNNABLE
- Жив: true

```

[Maintenance-737-800] Starting maintenance for 737-800
[Maintenance-737-800] Checking engine systems...
[Maintenance-737-800] Testing avionics...
[Maintenance-737-800] Inspecting airframe...
[Maintenance-737-800] Checking fuel systems...
[Maintenance-737-800] Performing final checks...

```

*** Performing maintenance on 737-800 ***

```

Checking engine...
Inspecting wings and fuselage...
Testing avionics...
Maintenance complete!
[Maintenance-737-800] Maintenance completed for 737-800

```

Информация о потоке ПОСЛЕ завершения:

- Состояние: TERMINATED
- Жив: false

=====

ПРИМЕР 5: Запуск множественных потоков одного типа

=====

Демонстрация: симуляция работы аэропорта с несколькими рейсами

```
[Flight-737-700-to-Paris] Starting flight simulation for 737-700
[Flight-737-700-to-Paris] Pre-flight checks...
[Flight-A319-to-Berlin] Starting flight simulation for A319
[Flight-A319-to-Berlin] Pre-flight checks...
[Flight-737-900-to-Rome] Starting flight simulation for 737-900
[Flight-737-900-to-Rome] Pre-flight checks...
[Flight-737-700-to-Paris] Starting engines...
```

=== Starting Boeing 737-700 ===

```
Engine Unknown starting...
Aircraft systems initialized.
[Flight-A319-to-Berlin] Starting engines...
```

=== Starting Airbus A319 ===

```
Engine Unknown starting...
Aircraft systems initialized.
[Flight-737-900-to-Rome] Starting engines...
```

=== Starting Boeing 737-900 ===

```
Engine Unknown starting...
Aircraft systems initialized.
[Flight-737-700-to-Paris] Taking off to Paris
```

```
>>> 737-700 is taking off...
Engine Unknown quick starting!
Accelerating on runway...
Lifting off!
[Flight-737-900-to-Rome] Taking off to Rome
```

```
>>> 737-900 is taking off...
Engine Unknown quick starting!
Accelerating on runway...
Lifting off!
[Flight-A319-to-Berlin] Taking off to Berlin
```

```
>>> A319 is taking off...
Engine Unknown quick starting!
Accelerating on runway...
```


Lifting off!

[Flight-737-700-to-Paris] Climbing to cruise altitude...

Flying at 10000 meters altitude.

[Flight-737-900-to-Rome] Climbing to cruise altitude...

Flying at 10000 meters altitude.

[Flight-A319-to-Berlin] Climbing to cruise altitude...

Flying at 10000 meters altitude.

[Flight-737-700-to-Paris] Cruising at 10,000m to Paris

[Flight-737-700-to-Paris] Flight hour 1 completed

[Flight-A319-to-Berlin] Cruising at 10,000m to Berlin

[Flight-A319-to-Berlin] Flight hour 1 completed

[Flight-737-900-to-Rome] Cruising at 10,000m to Rome

[Flight-737-900-to-Rome] Flight hour 1 completed

[Flight-737-700-to-Paris] Flight hour 2 completed

[Flight-A319-to-Berlin] Flight hour 2 completed

[Flight-737-900-to-Rome] Flight hour 2 completed

[Flight-A319-to-Berlin] Flight hour 3 completed

[Flight-737-700-to-Paris] Flight hour 3 completed

[Flight-737-900-to-Rome] Flight hour 3 completed

[Flight-737-700-to-Paris] Descending for landing...

[Flight-A319-to-Berlin] Descending for landing...

[Flight-737-900-to-Rome] Descending for landing...

[Flight-A319-to-Berlin] Landing at Berlin

[Flight-737-700-to-Paris] Landing at Paris

<<< A319 is landing...

Descending...

<<< 737-700 is landing...

Descending...

Touchdown!

Taxiing to gate.

Touchdown!

Taxiing to gate.

[Flight-737-900-to-Rome] Landing at Rome

<<< 737-900 is landing...

Descending...

Touchdown!

Taxiing to gate.

[Flight-A319-to-Berlin] Shutting down engines...

=== Stopping Airbus A319 ===

Engine Unknown stopped.

Aircraft systems shut down.

[Flight-A319-to-Berlin] Flight completed! Total hours: 5

[Flight-737-700-to-Paris] Shutting down engines...

```
=== Stopping Boeing 737-700 ===
Engine Unknown stopped.
Aircraft systems shut down.
[Flight-737-700-to-Paris] Flight completed! Total hours: 5
[Flight-737-900-to-Rome] Shutting down engines...
```

```
=== Stopping Boeing 737-900 ===
Engine Unknown stopped.
Aircraft systems shut down.
[Flight-737-900-to-Rome] Flight completed! Total hours: 5
```

✓ Все рейсы завершены

```
=====
ИТОГИ ДЕМОНСТРАЦИИ
=====
```

Продemonстрированные концепции:

- ✓ 1. Создание потоков через наследование от Thread
- ✓ 2. Переопределение метода run()
- ✓ 3. Запуск потоков через start()
- ✓ 4. Параллельное выполнение нескольких потоков
- ✓ 5. Использование join() для ожидания завершения
- ✓ 6. Именованное потоков для идентификации
- ✓ 7. Получение информации о состоянии потоков
- ✓ 8. Обработка InterruptedException

Главный поток (main) завершен!

```
=====
ЗАДАНИЕ 2: ДЕМОНСТРАЦИЯ РАБОТЫ С ИНТЕРФЕЙСОМ RUNNABLE
=====
```

Создано 3 самолета для демонстрации работы с Runnable

```
=====
ПРИМЕР 1: Базовое использование Runnable
=====
```

Демонстрация: создание задач через реализацию Runnable
Запускаем задачи заправки и посадки параллельно

```
[Board-Thread-1] Starting boarding for A320 at Gate-H
[Refuel-Thread-1] Starting refueling for 737-800 at Station-77
[Board-Thread-1] Opening boarding gate...
[Refuel-Thread-1] Connecting fuel hose...
```

```

[Board-Thread-1] Boarding group: 30/180 passengers on A320
[Refuel-Thread-1] Checking fuel system...
[Refuel-Thread-1] Refueling 737-800: 1000/5000 liters
[Board-Thread-1] Boarding group: 60/180 passengers on A320
[Refuel-Thread-1] Refueling 737-800: 2000/5000 liters
[Refuel-Thread-1] Refueling 737-800: 3000/5000 liters
[Board-Thread-1] Boarding group: 90/180 passengers on A320
[Refuel-Thread-1] Refueling 737-800: 4000/5000 liters
[Board-Thread-1] Boarding group: 120/180 passengers on A320
[Refuel-Thread-1] Refueling 737-800: 5000/5000 liters
[Board-Thread-1] Boarding group: 150/180 passengers on A320
[Refuel-Thread-1] Disconnecting fuel hose...
[Refuel-Thread-1] ✓ Refueling completed for 737-800 (5000.0 liters)
[Board-Thread-1] Boarding group: 180/180 passengers on A320
[Board-Thread-1] Checking cabin, securing doors...
[Board-Thread-1] Safety briefing in progress...
[Board-Thread-1] ✓ Boarding completed for A320 (180 passengers)

```

✓ Обе задачи завершены

```

=====
ПРИМЕР 2: Повторное использование задачи
=====

```

Демонстрация: одна задача может быть запущена в нескольких потоках
 Запускаем одну задачу предполетной проверки в 3 потоках

```

[Check-Thread-1] Starting pre-flight check for 777-300ER by Chief-Inspector
[Check-Thread-2] Starting pre-flight check for 777-300ER by Chief-Inspector
[Check-Thread-3] Starting pre-flight check for 777-300ER by Chief-Inspector
[Check-Thread-3] External inspection of 777-300ER
[Check-Thread-1] External inspection of 777-300ER
[Check-Thread-2] External inspection of 777-300ER
[Check-Thread-1] - Checking fuselage and wings
[Check-Thread-2] - Checking fuselage and wings
[Check-Thread-3] - Checking fuselage and wings
[Check-Thread-1] - Inspecting landing gear
[Check-Thread-2] - Inspecting landing gear
[Check-Thread-3] - Inspecting landing gear
[Check-Thread-1] Engine inspection
[Check-Thread-3] Engine inspection
[Check-Thread-2] Engine inspection
[Check-Thread-1] - Testing engine systems
[Check-Thread-3] - Testing engine systems
[Check-Thread-2] - Testing engine systems
[Check-Thread-1] Avionics check
[Check-Thread-3] Avionics check
[Check-Thread-2] Avionics check

```

```

[Check-Thread-3] - Testing navigation systems
[Check-Thread-1] - Testing navigation systems
[Check-Thread-2] - Testing navigation systems
[Check-Thread-2] - Checking communication equipment
[Check-Thread-1] - Checking communication equipment
[Check-Thread-3] - Checking communication equipment
[Check-Thread-1] Fuel system verification
[Check-Thread-3] Fuel system verification
[Check-Thread-2] Fuel system verification
[Check-Thread-3] Hydraulic systems check
[Check-Thread-2] Hydraulic systems check
[Check-Thread-1] Hydraulic systems check
[Check-Thread-3] Final safety checks
[Check-Thread-2] Final safety checks
[Check-Thread-1] Final safety checks
[Check-Thread-1] ✓ Pre-flight check completed for 777-300ER - CLEARED FOR TAKEOFF
[Check-Thread-2] ✓ Pre-flight check completed for 777-300ER - CLEARED FOR TAKEOFF
[Check-Thread-3] ✓ Pre-flight check completed for 777-300ER - CLEARED FOR TAKEOFF

```

✓ Все проверки завершены

=====

ПРИМЕР 3: Анонимный класс Runnable

=====

Демонстрация: создание Runnable через анонимный класс

```

[Anonymous-Thread] Starting anonymous task for 737-800
[Anonymous-Thread] Step 1 of 3
[Anonymous-Thread] Step 2 of 3
[Anonymous-Thread] Step 3 of 3
[Anonymous-Thread] ✓ Anonymous task completed

```

=====

ПРИМЕР 4: Лямбда-выражения для Runnable

=====

Демонстрация: современный способ создания Runnable через лямбда

```

[Lambda-Thread-1] Lambda task started
[Lambda-Thread-2] Preparing flight to New York
[Lambda-Thread-2] Flight plan for 777-300ER approved
[Lambda-Thread-1] Processing A320
[Lambda-Thread-2] ✓ Ready for departure to New York
[Lambda-Thread-1] ✓ Lambda task completed

```

✓ Все лямбда-задачи завершены

=====

ПРИМЕР 5: Комплексная подготовка к полету

=====

Демонстрация: параллельное выполнение всех задач подготовки

Starting all preparation tasks for 737-800...

[Refuel-B737] Starting refueling for 737-800 at Station-A1
[Refuel-B737] Connecting fuel hose...
[Board-B737] Starting boarding for 737-800 at Gate-B5
[Board-B737] Opening boarding gate...
[Check-B737] Starting pre-flight check for 737-800 by Captain-Smith
[Check-B737] External inspection of 737-800
[Board-B737] Boarding group: 30/189 passengers on 737-800
[Refuel-B737] Checking fuel system...
[Check-B737] - Checking fuselage and wings
[Refuel-B737] Refueling 737-800: 1000/6000 liters
[Check-B737] - Inspecting landing gear
[Board-B737] Boarding group: 60/189 passengers on 737-800
[Refuel-B737] Refueling 737-800: 2000/6000 liters
[Check-B737] Engine inspection
[Board-B737] Boarding group: 90/189 passengers on 737-800
[Refuel-B737] Refueling 737-800: 3000/6000 liters
[Check-B737] - Testing engine systems
[Refuel-B737] Refueling 737-800: 4000/6000 liters
[Board-B737] Boarding group: 120/189 passengers on 737-800
[Check-B737] Avionics check
[Refuel-B737] Refueling 737-800: 5000/6000 liters
[Board-B737] Boarding group: 150/189 passengers on 737-800
[Check-B737] - Testing navigation systems
[Refuel-B737] Refueling 737-800: 6000/6000 liters
[Check-B737] - Checking communication equipment
[Refuel-B737] Disconnecting fuel hose...
[Board-B737] Boarding group: 180/189 passengers on 737-800
[Check-B737] Fuel system verification
[Refuel-B737] ✓ Refueling completed for 737-800 (6000.0 liters)
[Check-B737] Hydraulic systems check
[Board-B737] Boarding group: 189/189 passengers on 737-800
[Check-B737] Final safety checks
[Board-B737] Checking cabin, securing doors...
[Check-B737] ✓ Pre-flight check completed for 737-800 - CLEARED FOR TAKEOFF
[Board-B737] Safety briefing in progress...
[Board-B737] ✓ Boarding completed for 737-800 (189 passengers)

✓ Все задачи подготовки завершены - самолет готов к вылету!

=====

ПРИМЕР 6: Сравнение подходов Thread vs Runnable

Подход 1: Наследование от Thread (из Задания 1)

Подход 2: Реализация Runnable (Задание 2)

```
[Maintenance-A320] Starting maintenance for A320
[Maintenance-A320] Checking engine systems...
[Runnable-Approach] Starting refueling for A320 at Station-60
[Runnable-Approach] Connecting fuel hose...
[Runnable-Approach] Checking fuel system...
[Runnable-Approach] Refueling A320: 1000/4500 liters
[Maintenance-A320] Testing avionics...
[Runnable-Approach] Refueling A320: 2000/4500 liters
[Runnable-Approach] Refueling A320: 3000/4500 liters
[Maintenance-A320] Inspecting airframe...
[Runnable-Approach] Refueling A320: 4000/4500 liters
[Runnable-Approach] Refueling A320: 4500/4500 liters
[Runnable-Approach] Disconnecting fuel hose...
[Maintenance-A320] Checking fuel systems...
[Runnable-Approach] ✓ Refueling completed for A320 (4500.0 liters)
[Maintenance-A320] Performing final checks...
```

```
*** Performing maintenance on A320 ***
Checking engine...
Inspecting wings and fuselage...
Testing avionics...
Maintenance complete!
[Maintenance-A320] Maintenance completed for A320
```

✓ Оба подхода завершены

ИТОГИ ДЕМОНСТРАЦИИ

Продemonстрированные концепции:

- ✓ 1. Реализация интерфейса Runnable
- ✓ 2. Создание потоков через new Thread(Runnable)
- ✓ 3. Повторное использование одной задачи в разных потоках
- ✓ 4. Анонимные классы для Runnable
- ✓ 5. Лямбда-выражения (современный подход)
- ✓ 6. Параллельное выполнение множества задач
- ✓ 7. Сравнение Thread vs Runnable

Главный поток (main) завершен!

=====

ЗАДАНИЕ 3: ДЕМОНСТРАЦИЯ РАБОТЫ С ПРИОРИТЕТАМИ ПОТОКОВ

=====

Диапазон приоритетов в Java:

```
MIN_PRIORITY  = 1 (минимальный)
NORM_PRIORITY = 5 (по умолчанию)
MAX_PRIORITY  = 10 (максимальный)
```

=====

ПРИМЕР 1: Базовая установка приоритетов

=====

Демонстрация: создание потоков с разными приоритетами

Приоритеты установлены:

```
Emergency-Thread: 10
Maintenance-Thread: 5
Logging-Thread: 1
```

```
[Emergency-Thread] EMERGENCY! Priority: 10
[Logging-Thread] Starting background logging. Priority: 1
[Maintenance-Thread] Starting routine maintenance. Priority: 5
[Emergency-Thread] 737-800 - Engine failure
[Logging-Thread] 777-300ER - Logging 5 records
[Maintenance-Thread] A320 - Scheduled check
[Logging-Thread] 📝 Logging record 1/5 for 777-300ER
[Emergency-Thread] ⚠ Declaring emergency...
[Maintenance-Thread] 🔧 Checking systems...
[Emergency-Thread] ⚠ Clearing airspace...
[Logging-Thread] 📝 Logging record 2/5 for 777-300ER
[Maintenance-Thread] 🔧 Inspecting components...
[Emergency-Thread] ⚠ Preparing emergency landing...
[Emergency-Thread] ⚠ Emergency services on standby...
[Logging-Thread] 📝 Logging record 3/5 for 777-300ER
[Maintenance-Thread] 🔧 Testing equipment...
[Emergency-Thread] ⚠ Beginning emergency descent...
[Logging-Thread] 📝 Compressing data...
[Maintenance-Thread] 🔧 Replacing filters...
[Emergency-Thread] ⚠ Final approach...
[Logging-Thread] 📝 Logging record 4/5 for 777-300ER
[Emergency-Thread] ⚠ EMERGENCY LANDING COMPLETE!
[Emergency-Thread] ✓ Emergency handled in 832ms
[Emergency-Thread] Aircraft and passengers safe!
```

[Maintenance-Thread]  Lubricating parts...
[Logging-Thread]  Logging record 5/5 for 777-300ER
[Maintenance-Thread]  Final inspection...
[Logging-Thread]  Finalizing log file...
[Maintenance-Thread]  Updating maintenance log...
[Logging-Thread] ✓ Logging completed in 1281ms
[Logging-Thread] Total records: 5
[Maintenance-Thread] ✓ Routine maintenance completed in 1309ms

✓ Все задачи завершены

=====

ПРИМЕР 2: Влияние приоритетов на время выполнения

=====

Демонстрация: запуск одинаковых задач с разными приоритетами

Запуск трех одинаковых задач с разными приоритетами...

[High-Priority] Starting background logging. Priority: 10
[High-Priority] 737-800 - Logging 8 records
[High-Priority]  Logging record 1/8 for 737-800
[Normal-Priority] Starting background logging. Priority: 5
[Normal-Priority] A320 - Logging 8 records
[Normal-Priority]  Logging record 1/8 for A320
[Low-Priority] Starting background logging. Priority: 1
[Low-Priority] 777-300ER - Logging 8 records
[Low-Priority]  Logging record 1/8 for 777-300ER
[Normal-Priority]  Logging record 2/8 for A320
[High-Priority]  Logging record 2/8 for 737-800
[Low-Priority]  Logging record 2/8 for 777-300ER
[Low-Priority]  Logging record 3/8 for 777-300ER
[Normal-Priority]  Logging record 3/8 for A320
[High-Priority]  Logging record 3/8 for 737-800
[Normal-Priority]  Compressing data...
[Low-Priority]  Compressing data...
[High-Priority]  Compressing data...
[Normal-Priority]  Logging record 4/8 for A320
[Low-Priority]  Logging record 4/8 for 777-300ER
[High-Priority]  Logging record 4/8 for 737-800
[Low-Priority]  Logging record 5/8 for 777-300ER
[Normal-Priority]  Logging record 5/8 for A320
[High-Priority]  Logging record 5/8 for 737-800


```

[Low-Priority] [ ] Logging record 6/8 for 777-300ER
[Normal-Priority] [ ] Logging record 6/8 for A320
[High-Priority] [ ] Logging record 6/8 for 737-800
[Low-Priority] [ ] Compressing data...
[Normal-Priority] [ ] Compressing data...
[High-Priority] [ ] Compressing data...
[Low-Priority] [ ] Logging record 7/8 for 777-300ER
[Normal-Priority] [ ] Logging record 7/8 for A320
[High-Priority] [ ] Logging record 7/8 for 737-800
[Low-Priority] [ ] Logging record 8/8 for 777-300ER
[Normal-Priority] [ ] Logging record 8/8 for A320
[High-Priority] [ ] Logging record 8/8 for 737-800
[Low-Priority] [ ] Finalizing log file...
[Normal-Priority] [ ] Finalizing log file...
[High-Priority] [ ] Finalizing log file...
[Low-Priority] ✓ Logging completed in 1990ms
[Low-Priority] Total records: 8
[Normal-Priority] ✓ Logging completed in 1993ms
[Normal-Priority] Total records: 8
[High-Priority] ✓ Logging completed in 2003ms
[High-Priority] Total records: 8

```

--- Результаты ---

```

High Priority (10):    2003ms
Normal Priority (5):   1993ms
Low Priority (1):      1990ms

```

Примечание: Разница может быть незначительной из-за особенностей планировщика ОС

=====

ПРИМЕР 3: Динамическое изменение приоритета

=====

Демонстрация: изменение приоритета работающего потока

Начальный приоритет: 5

Установлен MIN_PRIORITY: 1

```

[Dynamic-Priority] Starting background logging. Priority: 1
[Dynamic-Priority] 737-800 - Logging 6 records
[Dynamic-Priority] [ ] Logging record 1/6 for 737-800
[Dynamic-Priority] [ ] Logging record 2/6 for 737-800
[Dynamic-Priority] [ ] Logging record 3/6 for 737-800
[Dynamic-Priority] [ ] Compressing data...
[Dynamic-Priority] [ ] Logging record 4/6 for 737-800

```

[Dynamic-Priority]  Logging record 5/6 for 737-800

>>> Повышаем приоритет до MAX_PRIORITY

Новый приоритет: 10

[Dynamic-Priority]  Logging record 6/6 for 737-800

[Dynamic-Priority]  Compressing data...

[Dynamic-Priority]  Finalizing log file...

[Dynamic-Priority] ✓ Logging completed in 1581ms

[Dynamic-Priority] Total records: 6

✓ Задача завершена

=====

ПРИМЕР 4: Использование пользовательских значений приоритетов

=====

Демонстрация: установка произвольных значений от 1 до 10

Created Task-1 with priority 1

Created Task-2 with priority 3

Created Task-3 with priority 5

Created Task-4 with priority 7

Created Task-5 with priority 10

Запуск всех потоков...

[Task-1] Running with priority 1

[Task-3] Running with priority 5

[Task-4] Running with priority 7

[Task-2] Running with priority 3

[Task-5] Running with priority 10

[Task-4] Completed

[Task-5] Completed

[Task-3] Completed

[Task-2] Completed

[Task-1] Completed

✓ Все задачи завершены

=====

ПРИМЕР 5: Практический сценарий - Управление воздушным движением

=====

Демонстрация: реалистичное использование приоритетов

Приоритеты операций:

 EMERGENCY: 10 (КРИТИЧНО)

 Routine-Ops: 5 (обычная)
 Refuel-Ops: 5 (обычная)
 Background-Log-1: 1 (фон)
 Background-Log-2: 1 (фон)

[EMERGENCY] EMERGENCY! Priority: 10
 [Routine-Ops] Starting routine maintenance. Priority: 5
 [Routine-Ops] A320 - Pre-flight inspection
 [Routine-Ops]  Checking systems...
 [EMERGENCY] 737-800 - Medical emergency
 [EMERGENCY]  Declaring emergency...
 [Refuel-Ops] Starting refueling for 777-300ER at Station-19
 [Refuel-Ops] Connecting fuel hose...
 [Background-Log-1] Starting background logging. Priority: 1
 [Background-Log-2] Starting background logging. Priority: 1
 [Background-Log-2] A320 - Logging 4 records
 [Background-Log-2]  Logging record 1/4 for A320
 [Background-Log-1] 737-800 - Logging 4 records
 [Background-Log-1]  Logging record 1/4 for 737-800
 [EMERGENCY]  Clearing airspace...
 [Routine-Ops]  Inspecting components...
 [Background-Log-2]  Logging record 2/4 for A320
 [Background-Log-1]  Logging record 2/4 for 737-800
 [EMERGENCY]  Preparing emergency landing...
 [EMERGENCY]  Emergency services on standby...
 [Background-Log-1]  Logging record 3/4 for 737-800
 [Background-Log-2]  Logging record 3/4 for A320
 [Routine-Ops]  Testing equipment...
 [EMERGENCY]  Beginning emergency descent...
 [Refuel-Ops] Checking fuel system...
 [Background-Log-1]  Compressing data...
 [Background-Log-2]  Compressing data...
 [Routine-Ops]  Replacing filters...
 [EMERGENCY]  Final approach...
 [Background-Log-2]  Logging record 4/4 for A320
 [Background-Log-1]  Logging record 4/4 for 737-800
 [Refuel-Ops] Refueling 777-300ER: 1000/8000 liters
 [EMERGENCY]  EMERGENCY LANDING COMPLETE!
 [EMERGENCY] ✓ Emergency handled in 821ms
 [EMERGENCY] Aircraft and passengers safe!
 [Routine-Ops]  Lubricating parts...
 [Background-Log-2]  Finalizing log file...

```
[Background-Log-1] 📄 Finalizing log file...
[Routine-Ops] 🔧 Final inspection...
[Background-Log-2] ✓ Logging completed in 1074ms
[Background-Log-2] Total records: 4
[Background-Log-1] ✓ Logging completed in 1074ms
[Background-Log-1] Total records: 4
[Routine-Ops] 🔧 Updating maintenance log...
[Refuel-Ops] Refueling 777-300ER: 2000/8000 liters
[Routine-Ops] ✓ Routine maintenance completed in 1303ms
[Refuel-Ops] Refueling 777-300ER: 3000/8000 liters
[Refuel-Ops] Refueling 777-300ER: 4000/8000 liters
[Refuel-Ops] Refueling 777-300ER: 5000/8000 liters
[Refuel-Ops] Refueling 777-300ER: 6000/8000 liters
[Refuel-Ops] Refueling 777-300ER: 7000/8000 liters
[Refuel-Ops] Refueling 777-300ER: 8000/8000 liters
[Refuel-Ops] Disconnecting fuel hose...
[Refuel-Ops] ✓ Refueling completed for 777-300ER (8000.0 liters)
```

✓ Все операции завершены

=====

ИТОГИ ДЕМОНСТРАЦИИ

=====

Продemonстрированные концепции:

- ✓ 1. Установка приоритетов через `setPriority()`
- ✓ 2. Использование констант `MIN/NORM/MAX_PRIORITY`
- ✓ 3. Пользовательские значения приоритетов (1-10)
- ✓ 4. Динамическое изменение приоритета
- ✓ 5. Влияние приоритетов на выполнение
- ✓ 6. Практические сценарии использования

Главный поток (main) завершен!

=====

ЗАДАНИЕ 4: ДЕМОНСТРАЦИЯ СИНХРОНИЗАЦИИ ПОТОКОВ

=====

=====

ПРИМЕР 1: Демонстрация проблемы Race Condition

=====

Проблема: несколько потоков изменяют общий ресурс без синхронизации

Запуск 10 потоков, каждый увеличивает счетчик 1000 раз

Ожидаемый результат: 10000

Фактический результат: 10000

✗ Результат НЕВЕРНЫЙ из-за race condition!

Потоки одновременно изменяли счетчик без синхронизации

=====

ПРИМЕР 2: Решение проблемы через synchronized

=====

Решение: использование synchronized для защиты критической секции

Запуск 10 потоков, каждый увеличивает счетчик 1000 раз

Ожидаемый результат: 10000

Фактический результат: 10000

✓ Результат ВЕРНЫЙ благодаря synchronized!

Синхронизация гарантирует, что только один поток изменяет счетчик

=====

ПРИМЕР 3: Синхронизация доступа к общему ресурсу

=====

Сценарий: несколько самолетов используют одну взлетно-посадочную полосу

[Boeing-737-Thread] Boeing-737 requesting Runway-01 for taking off

[Boeing-737-Thread] ✓ Boeing-737 acquired Runway-01 for taking off

[Boeing-737-Thread] >>> Boeing-737 is taking off on Runway-01

[Boeing-737-Thread] <<< Boeing-737 completed taking off

[Boeing-737-Thread] ✓ Boeing-737 released Runway-01

[Airbus-A320-Thread] Airbus-A320 requesting Runway-01 for landing

[Airbus-A320-Thread] ✓ Airbus-A320 acquired Runway-01 for landing

[Boeing-777-Thread] Boeing-777 requesting Runway-01 for taking off

[Boeing-777-Thread] Boeing-777 waiting... (occupied by Airbus-A320)

[Airbus-A380-Thread] Airbus-A380 requesting Runway-01 for landing

[Airbus-A380-Thread] Airbus-A380 waiting... (occupied by Airbus-A320)

[Airbus-A320-Thread] >>> Airbus-A320 is landing on Runway-01

[Airbus-A320-Thread] <<< Airbus-A320 completed landing

[Airbus-A320-Thread] ✓ Airbus-A320 released Runway-01

[Boeing-777-Thread] ✓ Boeing-777 acquired Runway-01 for taking off

[Boeing-777-Thread] >>> Boeing-777 is taking off on Runway-01

[Boeing-777-Thread] <<< Boeing-777 completed taking off

[Boeing-777-Thread] ✓ Boeing-777 released Runway-01

[Airbus-A380-Thread] ✓ Airbus-A380 acquired Runway-01 for landing

[Airbus-A380-Thread] >>> Airbus-A380 is landing on Runway-01

[Airbus-A380-Thread] <<< Airbus-A380 completed landing

[Airbus-A380-Thread] ✓ Airbus-A380 released Runway-01

✓ Все операции на полосе завершены безопасно

Синхронизация предотвратила одновременное использование полосы

=====

ПРИМЕР 4: Паттерн Producer-Consumer

=====

Сценарий: поставщики заправляют резервуар, самолеты забирают топливо

[Boeing-737-Thread] Boeing-737 waiting for fuel... Tank Main-Tank has only 0 liters (need 2500)

[Airbus-A320-Thread] Airbus-A320 waiting for fuel... Tank Main-Tank has only 0 liters (need 3500)

[Supplier-1-Thread] 📦 Supplier-1 refilled Tank Main-Tank with 3000 liters

[Supplier-1-Thread] Tank Main-Tank level: 3000/10000 liters

[Boeing-737-Thread] ✂ Boeing-737 withdrew 2500 liters from Tank Main-Tank

[Boeing-737-Thread] Tank Main-Tank level: 500/10000 liters

[Airbus-A320-Thread] Airbus-A320 waiting for fuel... Tank Main-Tank has only 500 liters (need 3500)

[Supplier-2-Thread] 📦 Supplier-2 refilled Tank Main-Tank with 4000 liters

[Supplier-2-Thread] Tank Main-Tank level: 4500/10000 liters

[Airbus-A320-Thread] ✂ Airbus-A320 withdrew 3500 liters from Tank Main-Tank

[Airbus-A320-Thread] Tank Main-Tank level: 1000/10000 liters

[Supplier-1-Thread] 📦 Supplier-1 refilled Tank Main-Tank with 3000 liters

[Supplier-1-Thread] Tank Main-Tank level: 4000/10000 liters

[Boeing-737-Thread] ✂ Boeing-737 withdrew 2500 liters from Tank Main-Tank

[Boeing-737-Thread] Tank Main-Tank level: 1500/10000 liters

[Supplier-2-Thread] 📦 Supplier-2 refilled Tank Main-Tank with 4000 liters

[Supplier-2-Thread] Tank Main-Tank level: 5500/10000 liters

[Supplier-1-Thread] 📦 Supplier-1 refilled Tank Main-Tank with 3000 liters

[Supplier-1-Thread] Tank Main-Tank level: 8500/10000 liters

[Airbus-A320-Thread] ✂ Airbus-A320 withdrew 3000 liters from Tank Main-Tank

[Airbus-A320-Thread] Tank Main-Tank level: 5500/10000 liters

✓ Все операции с топливом завершены

Финальный уровень в резервуаре: 5500 литров

wait/notify обеспечили корректную координацию

=====

ПРИМЕР 5: Комплексная синхронизация операций аэропорта

=====

Сценарий: координация взлетов, посадок и заправки

[Supplier-Thread] 📦 Initial-Supply refilled Tank Airport-Tank with 13000 liters

[Supplier-Thread] Tank Airport-Tank level: 13000/15000 liters

Запуск комплексных операций...

[Flight-AA101] ✂ Flight-AA101 withdrew 4000 liters from Tank Airport-Tank

[Flight-BA202] Flight-BA202 requesting Runway-02 for landing

```

[Flight-BA202] ✓ Flight-BA202 acquired Runway-02 for landing
[Flight-BA202] >>> Flight-BA202 is landing on Runway-02
[Flight-AA101] Tank Airport-Tank level: 9000/15000 liters
[Flight-LH303] ✂ Flight-LH303 withdrew 4500 liters from Tank Airport-Tank
[Flight-LH303] Tank Airport-Tank level: 4500/15000 liters
[Flight-BA202] <<< Flight-BA202 completed landing
[Flight-BA202] ✓ Flight-BA202 released Runway-02
[Flight-AA101] Flight-AA101 requesting Runway-02 for taking off
[Flight-AA101] ✓ Flight-AA101 acquired Runway-02 for taking off
[Flight-AA101] >>> Flight-AA101 is taking off on Runway-02
[Flight-BA202] ✂ Flight-BA202 withdrew 3500 liters from Tank Airport-Tank
[Flight-BA202] Tank Airport-Tank level: 1000/15000 liters
[Flight-AA101] <<< Flight-AA101 completed taking off
[Flight-AA101] ✓ Flight-AA101 released Runway-02
[Flight-LH303] Flight-LH303 requesting Runway-02 for taking off
[Flight-LH303] ✓ Flight-LH303 acquired Runway-02 for taking off
[Flight-LH303] >>> Flight-LH303 is taking off on Runway-02
[Flight-LH303] <<< Flight-LH303 completed taking off
[Flight-LH303] ✓ Flight-LH303 released Runway-02

```

✓ Все комплексные операции завершены успешно

Синхронизация обеспечила безопасную координацию:

- Взлетно-посадочная полоса использовалась по очереди
- Топливо распределялось корректно
- Не было конфликтов между потоками

```

=====
ИТОГИ ДЕМОНСТРАЦИИ
=====

```

Продemonстрированные концепции:

- ✓ 1. Проблема race condition и ее последствия
- ✓ 2. Использование synchronized для защиты критических секций
- ✓ 3. Методы wait() и notify() для координации потоков
- ✓ 4. Паттерн Producer-Consumer
- ✓ 5. Взаимное исключение при доступе к общим ресурсам
- ✓ 6. Практические сценарии синхронизации

Главный поток (main) завершен!

Выводы

В ходе выполнения лабораторной работы были изучены базовые принципы и механизмы многопоточного программирования в Java. Были рассмотрены и

реализованы два основных способа создания потоков: путём наследования от класса Thread и через реализацию интерфейса Runnable. Практика показала, что подход с использованием Runnable является более гибким и предпочтительным в современных приложениях.

В работе также исследовано назначение приоритетов потокам и подтверждено, что они служат лишь рекомендацией для планировщика ОС. Особое внимание было уделено проблемам параллелизма, таким как состояние гонки (race condition), и методам их решения с помощью синхронизации (synchronized, wait(), notify()).

Реализация системы управления авиационными операциями наглядно продемонстрировала практическое применение изученных концепций для координации задач, управления общими ресурсами и обеспечения потокобезопасности.